

Suponha que vários programas precisam utilizar o processador do computador para realizar operações aritméticas e lógicas e que o sistema operacional seja responsável por gerenciar o uso deste recurso. Cada programa possui:

- nome com até 64 caracteres, sem espaços.
- Tipo que pode ser tempo real (RT), sistema operacional (SO) ou de usuário (USER). Uma fila deverá ser mantida para cada tipo de programa, isto é, uma fila para programas de tempo real, uma para programas de sistema operacional e uma para programas de usuário.
- prioridade inteira.
- carga: número inteiro de operações que precisam ser feitas para finalizar o programa.
- último uso: número inteiro indicando em que momento o programa usou o processador pela última vez.
- iteração início: mais detalhes serão dados a seguir, mas este atributo define em que instante o programa entrará nas filas de uso do processador. Inicialmente é definido como -1.

O sistema operacional fica em um loop até que todos os programas sejam terminados. A cada iteração do loop a seguinte sequência de ações deve ser realizada:

1. **Adicionar novos programas nas filas:** A cada iteração, alguns programas começam a rodar, isto é, eles são adicionados pela primeira vez na fila do respectivo tipo. Nem toda iteração terá novos programas.
2. **Uso do processador:** um programa é selecionado para usar o processador e a carga é decrementada de uma instrução. Quando a carga chega a zero, o programa termina, seu nome é exibido na tela e ele é destruído. Se a carga for positiva, o atributo de último uso do processador deve ser definido como o número da iteração atual do loop e o programa é reintroduzido na fila.

Política de Seleção de Programas para Uso do Processador

- A fila de tempo real é uma fila simples implementada com *array circular*. Os programas desta fila têm prioridade sobre os outros tipos, mas dentre da fila todos possuem a mesma prioridade (o atributo de prioridade não é considerado para estes tipos de programas). Se a fila de tempo real não for vazia, o programa que usará o processador deverá vir desta fila. Programas só são selecionados das outras filas quando esta estiver vazia.
- As demais filas são filas de prioridade. Na fila de sistema operacional, processos são ordenados pelo atributo prioridade e, em caso de empate, pela menor carga. Isto é, programas pequenos são selecionados primeiro. Na fila de usuário, processos são ordenados pelo atributo prioridade e, em caso de empate, pelo instante de último uso. Isto é, programas que usaram o processador há mais tempo, são escolhidos primeiro. Em ambas as filas, em

caso de empate em todos os critérios, os programas devem ser selecionados por ordem alfabética (e.g., “chrome” teria mais prioridade que “word”).

- Programas são tirados destas duas filas de forma intercalada, isto é, se o último programa foi tirado da fila de sistema operacional, o próximo será tirado da fila de usuário e vice-versa. O primeiro é selecionado da fila de sistema operacional. Se uma das filas estiver vazia na sua vez, a oportunidade de uso do processador passa para a outra fila. Na próxima iteração, a vez volta para a outra fila. Por exemplo, se é a vez da fila de sistema operacional, mas ela está vazia, a vez será passada para a fila de usuário. Na iteração seguinte, será a vez da fila de sistema operacional, se ela não estiver vazia e a fila de tempo real continuar vazia.
 - **IMPORTANTE:** A prioridade para a fila de sistema operacional em relação à fila de usuário se dá APENAS no início do programa. Por exemplo, suponha que foi processado um programa da fila de sistema operacional. Depois, entrou um programa novo na fila de tempo real. Quando ele terminar, será a vez da fila de usuário, pois dentre as duas filas (de sistema operacional e de usuário), a última foi a de sistema operacional.
- Note que programas de tempo real podem ser adicionados a qualquer instante e devem ser executados até o final antes de selecionar um novo programa destas filas.

Entradas

Como ilustrado a seguir, o programa deve ler da entrada padrão um inteiro n representando o número de programas seguido das informações dos n programas. Para cada programa, é dado o nome, tipo, prioridade, carga e iteração de início da execução do programa.

```
6
Chrome USER 0 3 0
Word USER 0 2 0
Memchk SO 0 2 0
Procmap SO 0 1 0
Viewmax RT 0 2 0
ProMatrix RT 0 2 4
```

Saída

O programa deve mostrar na tela os nomes dos programas, na sequência em que eles forem encerrados, além do número da iteração em que eles terminaram. Para a entrada acima, a saída deve ser:

Viewmax 1 Procmap 2 ProMatrix 5 Memchk 8 Word 10 Chrome 11

A explicação detalhada da saída é dada no apêndice.

Regras

- Deve ser utilizado um heap para implementar a fila de prioridade.
- Deve ser utilizado um array circular para implementar a fila de real time.
- Todas as estruturas de dados para armazenar coleções (heap, array circular, etc.) devem ser opacas e genéricas.
- Erros no valgrind levarão a punições graves nas notas.

APÊNDICE

Simulação detalhada do programa para a entrada de exemplo.

Iteração 0

1) Atualização das filas: Entram na fila os programas com iteração de início igual a 0. O -1 após a barra indica a última iteração em que o programa usou o processador.

RT

- Viewmax RT 0 2 0 | -1

SO

- Memchk SO 0 2 0 | -1
- Procmap SO 0 1 0 | -1

USER

- Chrome USER 0 3 0 | -1
- Word USER 0 2 0 | -1

2) Execução de programa: A fila RT não é vazia, portanto, o programa vem desta fila: Viewmax. A carga é reduzida de 1 e o marcador de último uso do processador é atualizado. O programa é reintroduzido na fila.

RT

- Viewmax RT 0 1 0 | 0

SO

- Memchk SO 0 2 0 | -1
- Procmap SO 0 1 0 | -1

USER

- Chrome USER 0 3 0 | -1
- Word USER 0 2 0 | -1

Iteração 1

1) Atualização das filas: nenhum programa inicia a execução nesta iteração.

2) execução de programa: A fila RT não é vazia, portanto, o programa vem desta fila: Viewmax. A carga é reduzida de 1 e o programa termina. Os dados de Viewmax são exibidos na tela e o programa é destruído.

RT

SO

- Memchk SO 0 2 0 | -1
- Procmap SO 0 1 0 | -1

USER

- Chrome USER 1 3 0 | -1
- Word USER 0 2 0 | -1

Iteração 2

1) Atualização das filas: nenhum programa inicia a execução nesta iteração.

2) Execução de programa: A fila RT é vazia, então vamos executar programas das outras filas. Como descrito na especificação, primeiro selecionamos da fila de SO. Os dois programas têm a mesma prioridade e o Procmap tem menor carga, portanto é selecionado. A carga é decrementada e chega a zero. Os dados do programa são exibidos na tela e o programa é destruído.

RT

SO

- Memchk SO 0 2 0 | -1

USER

- Chrome USER 0 3 0 | -1
- Word USER 0 2 0 | -1

Iteração 3

1) Atualização das filas: nenhum programa inicia a execução nesta iteração.

2) Execução de programa: A fila RT é vazia, então vamos executar programas das outras filas. Seguindo a regra de intercalação, o próximo programa vem da fila user. Chrome tem maior prioridade e é selecionado primeiro. A carga é decrementada e o último uso do processador é definido como a iteração 3. O programa é reintroduzido na fila.

RT

SO

- Memchk SO 0 2 0 | -1

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 2 0 | -1

Iteração 4

1) Atualização das filas: o programa ProMatrix inicia a execução.

RT

- ProMatrix RT 0 2 4 | -1

SO

- Memchk SO 0 2 0 | -1

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 2 0 | -1

2) Execução de programa: O programa ProMatrix é selecionado da fila RT:

RT

- ProMatrix RT 0 1 4 | 4

SO

- Memchk SO 0 2 0 | -1

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 2 0 | -1

Iteração 5

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: O programa ProMatrix é selecionado da fila RT. O programa termina, seus dados são exibidos na tela e o programa é destruído.

RT

SO

- Memchk SO 0 2 0 | -1

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 2 0 | -1

Iteração 6

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: A fila RT está vazia e a lista USER foi usada mais recentemente que a lista SO. Portanto, o programa é selecionado da lista SO. Memchk tem sua carga reduzida e é reintroduzido na fila.

RT

SO

- Memchk SO 0 1 0 | 6

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 2 0 | -1

Iteração 7

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: A fila RT está vazia. O programa é selecionado da lista USER. Word tem mais prioridade e é selecionado. Após sua execução, é reintroduzido na fila.

RT

SO

- Memchk SO 0 1 0 | 6

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 1 0 | 7

Iteração 8

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: A fila RT está vazia. O programa é selecionado da lista SO. Memchk é selecionado e termina sua execução. Seus dados são exibidos na tela e o programa é destruído.

RT

SO

USER

- Chrome USER 0 2 0 | 3
- Word USER 0 1 0 | 7

Iteração 9

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: A fila RT está vazia. O programa é selecionado da lista USER. Chrome é selecionado, executa uma instrução e é reintroduzido na fila.

RT

SO

USER

- Chrome USER 0 1 0 | 9
- Word USER 0 1 0 | 7

Iteração 10

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: A fila RT está vazia. O programa seria selecionado da lista SO, mas ela está vazia. Portanto, a oportunidade passa para a lista USER. Word é selecionado, sua carga é reduzida e ele termina. Seu nome é exibido na tela e ele é destruído.

RT

SO

USER

- Chrome USER 0 1 0 | 9

Iteração 11

1) Atualização das filas: nenhum programa inicia a execução

2) Execução de programa: A fila RT está vazia. O programa seria selecionado da lista SO, mas ela está vazia. Portanto, a oportunidade passa para a lista USER. Chrome é selecionado, sua carga é reduzida e ele termina. Seus dados são exibidos na tela e o programa é destruído. Como as filas estão vazias, programa termina.

RT

SO

USER